Data Modelling in Power BI

AdventureWorks | From Prepared Tables to a Connected Model

After preparing the CSV data in Power Query, I moved to Model view. My next goal was to connect the tables so that report users could explore sales and returns by product, time, territory, and customer. This journal records what I learned in my guided Maven Analytics project.

Recognising facts and dimensions

In this project, tables named **Data** hold transactions and tables named **Lookup** describe business entities. These names help me navigate the project; the table's purpose and grain, not its name, determine its role.

Table	Role	What a row represents
Sales Data	Fact	One order line; quantity and dimension keys
Returns Data	Fact	Returned quantity for a date-product-territory combination
Calendar Lookup	Dimension	One calendar date
Customer Lookup	Dimension	One customer after key validation
Product Lookup	Dimension	One product
Product Subcategories Lookup	Dimension	One product subcategory
Product Categories Lookup	Dimension	One product category
Territory Lookup	Dimension	One sales territory / region

Six dimensions and two facts form the core model. Product Subcategories Lookup is a dimension even though it does not connect directly to a fact table. Measure and parameter tables used later are outside this eight-table inventory.

Why grain matters: 56,046 sales rows are not 56,046 orders. The source contains 25,164 distinct order numbers. Returns record quantities by date, product, and territory; they do not identify the original order or customer.

Reading My Model View

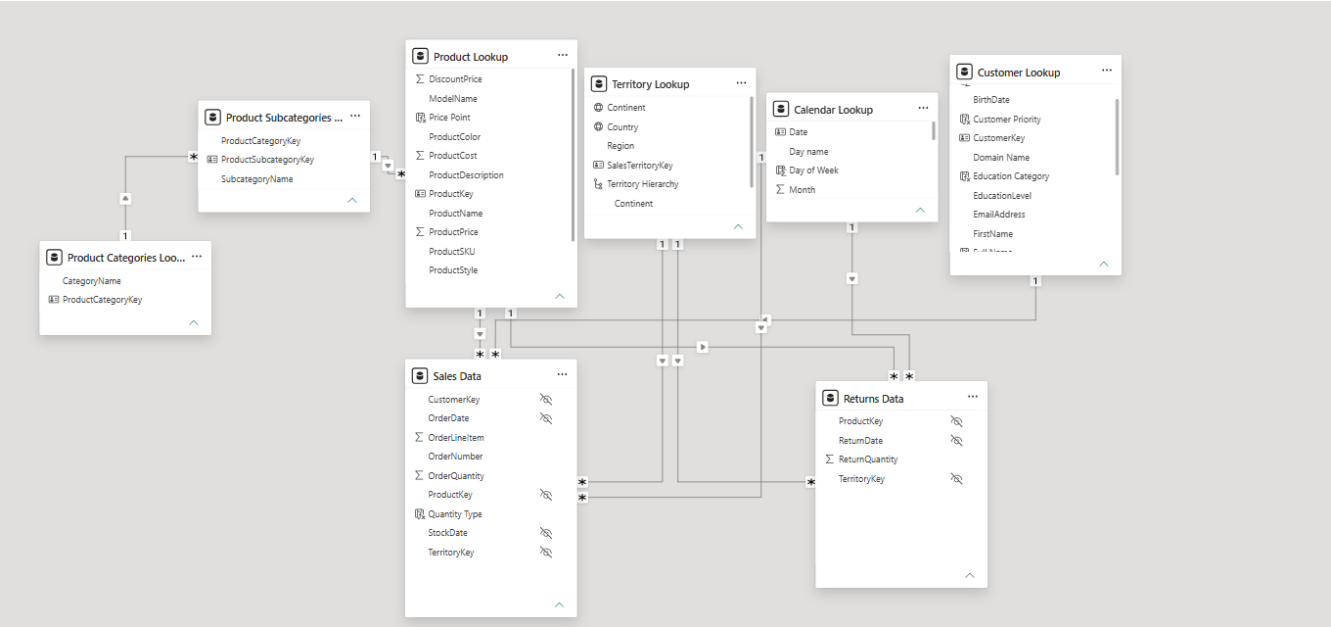


Figure 1. My supplied AdventureWorks Model view screenshot. It is a snapshot of the model, not a complete export of relationship properties.

How I read the diagram

Tables: lookup tables are arranged above and around Sales Data and Returns Data.
Relationship lines: identify connected table pairs.
1 and *: indicate the unique side and the repeating side of one-to-many relationships.
Arrow markers: indicate the direction in which filters propagate.

The structure visible in this project

Product, Calendar, and Territory provide shared dimensions for both fact tables. Customer relates to Sales. Product Categories connects through Product Subcategories to Product, creating the product snowflake branch.

There is no direct Sales-to-Returns relationship in this design. Shared dimensions allow corresponding measures to be compared at compatible levels without joining the two transaction tables together.

> **A practical limitation:** a Customer filter does not automatically filter Returns through single-direction relationships. Returns has no CustomerKey in the source. A customer-level return rate would therefore need additional attribution data or a different, explicitly justified model.

My modelling habit

I use the visual layout to understand the model, then open each relationship to verify the exact column pair, cardinality, filter direction, and active status. A tidy diagram alone does not establish that calculations are correct.

Primary Keys and Foreign Keys

A **primary key** identifies each row uniquely in a relational table. For my lookup tables, I use a single-column identifier that should be unique and nonblank. A **foreign key** is a reference to another table's identifier; repeated values are expected on the many side.

Dimension	Lookup identifier	Reference in a related table
Calendar Lookup	Date	Sales[OrderDate]; Returns[ReturnDate]
Customer Lookup	CustomerKey	Sales[CustomerKey]
Product Lookup	ProductKey	Sales[ProductKey]; Returns[ProductKey]
Territory Lookup	SalesTerritoryKey	Sales[TerritoryKey]; Returns[TerritoryKey]
Product Subcategories Lookup	ProductSubcategoryKey	Product[ProductSubcategoryKey]
Product Categories Lookup	ProductCategoryKey	Subcategories[ProductCategoryKey]

Table names are shortened in the right-hand column for readability. Matching columns do not need identical names: SalesTerritoryKey and TerritoryKey refer to the same territory identifier in these sources. A subcategory is both a dimension row and a row with a foreign key to its parent category.

What I mean by setting a key in Power BI

In Model view, I select the intended key column and inspect its Properties. If my course version exposes a **Key column / Is key** setting, it can be used to identify the column. That UI property is not a substitute for validation or a SQL PRIMARY KEY / FOREIGN KEY constraint. Availability and naming can differ by version.

The essential modelling step is placing the validated lookup key on the **1 side** of a one-to-many relationship. Creating a relationship does not generate or repair the fact's reference values: the matching keys must already exist.

Source-data issue to resolve: Customer CSV has 18,154 records, 18,152 distinct raw CustomerKey values, and one empty key. Repeated and empty identifiers must be inspected and handled before using the final query as a reliable customer dimension. The screenshot does not verify that cleanup.

Creating and Checking Relationships

My step-by-step workflow

1. Validate the dimension key. In Power Query, inspect null/empty keys and group by the key to identify repeated values. Investigate conflicts before retaining a record.
2. Align types. Both related columns should represent the same values using compatible types: whole-number IDs or date-only dates in this project.
3. Check references. Use a left anti join from a fact to its dimension to investigate unmatched keys.
4. Create the relationship. In Model view, drag the lookup key onto its fact reference, or use Manage relationships > New.
5. Review properties. Verify the column pair, 1:* cardinality, Single cross-filter direction from dimension to fact, and active status.
6. Test it. Add a dimension label and a simple quantity measure to a visual, then change a filter and confirm the result. [2]

One side	Many side / relationship to verify
Product Lookup[ProductKey]	Sales Data[ProductKey] and Returns Data[ProductKey]
Territory Lookup[SalesTerritoryKey]	Sales Data[TerritoryKey] and Returns Data[TerritoryKey]
Calendar Lookup[Date]	Sales Data[OrderDate] and Returns Data[ReturnDate]
Customer Lookup[CustomerKey]	Sales Data[CustomerKey]

Choosing the date role deliberately

Sales also contains StockDate. A relationship to OrderDate answers a different question from one to StockDate. I must confirm the intended active date role rather than relying on automatic detection. An inactive alternative can be used by an appropriate DAX measure when that date role is needed.

What I check in the diagram

A solid line represents an active relationship; a dashed line represents an inactive one. I start with a clear, single-direction dimension-to-fact design. Bidirectional filtering is not a general fix for missing results: with shared dimensions it can introduce ambiguous or unintended paths. [2]

My key lesson: establish relationships intentionally. Autodetection is a starting point, not evidence that the business meaning, grain, and filter behaviour are correct.

Star, Snowflake, and Merge Decisions

Star schema: dimensions around a fact

In a star design, descriptive dimensions connect directly to a central fact through one-to-many relationships. In my project, Sales connects to Product, Customer, Calendar, and Territory; Returns shares Product, Calendar, and Territory. The complete model therefore has two facts with shared dimensions. [1]

Snowflake: a normalised product dimension

Parent table (1)	Child table (*)	Matching key
Product Categories Lookup	Product Subcategories Lookup	ProductCategoryKey
Product Subcategories Lookup	Product Lookup	ProductSubcategoryKey
Product Lookup	Sales Data / Returns Data	ProductKey

Category and subcategory do not connect directly to the facts, but they are connected indirectly through Product. With active, single-direction relationships, selecting a category narrows its subcategories, then products, and finally the relevant transactions. This is a snowflake branch within the model. [1]

Why I did not merge every table into one

Flattening customer, product, and territory descriptions into each sales row repeats descriptive values. More importantly, merging Sales and Returns at an incompatible grain can duplicate rows and inflate quantities. Keeping separate facts and shared dimensions preserves their meanings and supports consistent comparisons.

The important trade-off

Relationships do **not** guarantee lower memory use or faster queries in every model. Compression, column cardinality, row counts, and query patterns matter. Merging Category and Subcategory into a single Product dimension is different from flattening every dimension into transaction data: a flattened Product dimension can simplify authoring and shorten filter paths. [1]

What I learned: the snowflake is useful for understanding this source structure. It is not automatically the most efficient production design. I would compare usability, model size, refresh time, and query behaviour before choosing a final architecture.

Filter Context, Row Context, and Filter Flow

My original notes used the phrase “filter row”. I now distinguish **filter context**, **row context**, and **filter flow**: they describe different behaviours. [3]

Concept	Meaning	Project example
Filter context	The filters under which an expression is evaluated	A measure evaluated for Year = 2022 and Country = Australia
Row context	The current row while an expression iterates or a calculated column is evaluated	Calculating a price value for the current Product row
Filter flow	Propagation across active model relationships	A Territory selection restricts matching Sales rows

Filter context: a measure responds to selections

Illustrative measure using the actual project column:

```
Units Sold = SUM('Sales Data'[OrderQuantity])
```

A Year filter on Calendar and a Country filter on Territory can both propagate to Sales. This measure then sums only records satisfying the combined filters. Visual rows and columns also create filter context; a table visual row does not automatically mean DAX row context. [2, 3]

Row context: one row at a time

Illustrative DAX calculated column in Product Lookup:

```
Discount Price Example = 'Product Lookup'[ProductPrice] * 0.90
```

Each product row supplies its own price. This is a DAX teaching example, not the original Power Query column from Part 1. Iterators such as SUMX also establish row context while evaluating their input rows. [3]

How they interact

Row context alone does not automatically filter an aggregation. CALCULATE can perform context transition, turning row context into filter context; it can also modify filters explicitly. I treat this as a bridge to the later DAX journal rather than assuming every row-by-row formula behaves like a report filter. [3, 4]

Test to remember: selecting a territory can filter both facts through shared dimensions. Selecting a customer normally filters Sales only in this model. The filter result depends on the available relationship paths.

Building a Date Hierarchy

A hierarchy defines a drill path from a broad level to more detail. It organises fields for report exploration; it does not create table relationships or new rows. [5]

A hierarchy based on prepared Calendar columns

Level	Calendar Lookup column	Purpose
1	Start of year	Group by year using a date-valued period start
2	Start of quarter	Expand into quarters within each year
3	Start of month	Expand into months within each quarter
4	Date	Inspect individual dates

This is an illustrative path using columns already discussed in Part 1, not a verified export of the final hierarchy. A label-based Year / Quarter / Month / Date path is also possible after creating the required labels and sort columns.

Steps I follow in Power BI Desktop

1. Expand Calendar Lookup in the Data pane.
2. Right-click Start of year and choose Create hierarchy.
3. Rename it Date Hierarchy.
4. Add Start of quarter, Start of month, and Date to the same hierarchy; order them from broad to detailed.
5. Add the hierarchy to a visual axis and practise drill down, drill up, and expand to the next level. [5]

Sorting and calendar checks

When I use Month name as a label, I sort it by the numeric Month column rather than alphabetically. I retain Year in the path so January from different years is not unintentionally combined. All hierarchy levels must belong to the same table. [1]

Calendar[Date] should be unique, nonblank, and contiguous. For classic DAX time intelligence, I mark Calendar Lookup as a date table and choose Date. This is separate from identifying a relationship key or creating a hierarchy. New calendar-based time intelligence has different configuration requirements. [6]

Week caution: I do not place Start of week inside a simple quarter/month hierarchy without checking boundary behaviour. Weeks can cross months, quarters, and years; a separate weekly drill path is clearer.

Building a Territory Hierarchy

Territory Lookup contains Continent, Country, and Region. These attributes provide a natural geographic drill path within a single table.

Level	Field	Example
1	Continent	North America
2	Country	United States
3	Region	Northwest

Steps I follow

1. Expand Territory Lookup in the Data pane.
2. Right-click Continent and choose Create hierarchy.
3. Name it Territory Hierarchy.
4. Add Country and Region, keeping that order below Continent.
5. Add the hierarchy and a quantity measure to a matrix or suitable chart, then explore each drill level. [5]

What the hierarchy lets me ask

I can begin with orders by continent, drill into a country, then inspect its regions. Because Territory relates to both facts, the same geographic selection can support Sales and Returns measures at compatible levels.

Important details from this dataset

The source has 10 territory records, 6 countries, and 3 continents. Some country and region labels are the same, while the United States has multiple region rows. This does not invalidate the hierarchy: the path reflects the territory structure in the supplied data.

SalesTerritoryKey is the relationship identifier. It need not be a hierarchy level and should not be treated as a numeric quantity to sum. I keep the readable geographic attributes available for grouping and consider hiding technical keys after validation.

> **Hierarchy versus relationship:** a hierarchy controls how I navigate levels within Territory. The relationship controls how those selections reach Sales and Returns. One does not replace the other.

What still needs verification

The screenshot shows a Territory Hierarchy object, but not every expanded level or property. The sequence above documents the intended reproducible setup. I would confirm the saved hierarchy in Desktop before describing all its levels as audited implementation details.

What I Learned and What I Verify

Modelling connects data preparation to analysis. I learned to define grain first, validate identifiers, choose clear relationship paths, and understand how filters change a result before building visuals.

Check	Project note
Table roles	6 dimensions; 2 facts
Dimension identifiers	Unique and nonblank in the final loaded queries
Customer cleanup	Source has repeated and empty keys; final query must be checked
Relationship pairs	Verify columns, 1:* cardinality, active status, and direction
Date roles	OrderDate versus StockDate must be deliberate
Totals and filters	Compare base totals, then test by date, territory, and product
Returns attribution	No order/customer linkage is available in the supplied Returns source
Hierarchies	Use same-table fields and verify ordering/drill behaviour
Performance	Measure results; do not assume more tables always use less memory

Evidence and documentation scope

Project-specific observations come from the supplied Model view screenshot, the CSV files, and my learning notes. CSV checks confirm 56,046 unique order-number/line-item combinations and 1,809 unique return-date/product/territory combinations. These support the described source grains but are not a full audit of the transformed model.

The property dialogs and complete saved hierarchy definitions were not supplied. Step-by-step instructions describe a reproducible approach, while version-dependent controls are explicitly qualified. DAX snippets are teaching examples. Maven Analytics provided the course and learning materials.

References | Microsoft Learn

[1] Star schema guidance

[2] Model relationships in Power BI Desktop

[3] DAX overview: context

[4] CALCULATE function

[5] End-to-end tutorial: create a date hierarchy

[6] Set and use date tables

References checked: 26 October 2026. Project author: iffaaulia.